Chapter 15: MULTISET

Now that you know:

```
set
lower_bound
upper_bound
```

Multiset becomes very easy.

What Problem Does Multiset Solve?

Normal Set:

```
set<int> s;
```

Stores:

```
Unique Elements Only
```

Example:

```
1 2 2 2 3
```

Stored:

```
1 2 3
```

Duplicates removed.

But sometimes duplicates matter.

Example:

```
1 2 2 2 3
```

You want to keep all values.

Use:

```
multiset<int> ms;
```

Stored:

```
1 2 2 2 3
```

Duplicates preserved.

Difference

Container	Duplicates
set	❌ No
multiset	✅ Yes

Internal Structure

Usually:

```
Red Black Tree
```

Same as set.

Therefore:

```
insert
erase
find
```

are:

```
O(log n)
```

Header

```
#include <set>
```

Declaration

```
multiset<int> ms;
```

Insert

```
ms.insert(10);
ms.insert(10);
ms.insert(10);
ms.insert(20);
```

Stored:

```
10 10 10 20
```

Print

```
for(auto x:ms)
{
    cout<<x<<" ";
}
```

Output:

```
10 10 10 20
```

Size

```
ms.size();
```

Output:

```
4
```

Count

Very useful.

```
ms.count(10);
```

Output:

```
3
```

Unlike set.

Complexity:

```
O(log n + count)
```

Find

```
auto it=ms.find(10);
```

Returns:

```
First occurrence of 10
```

Example:

```
10 10 10 20
^
```

Iterator points here.

Erase (Important)

Most students make mistakes here.

Input:

```
10 10 10 20
```

---

erase(value)

```
ms.erase(10);
```

Result:

```
20
```

ALL 10s removed.

This surprises many people.

Remove Only One Occurrence

Input:

```
10 10 10 20
```

```
auto it=ms.find(10);

ms.erase(it);
```

Result:

```
10 10 20
```

Only one removed.

Interview favorite.

---

lower_bound()

Works exactly like set.

Definition:

```
First Element >= x
```

Example:

```
1 2 2 2 5
```

```
ms.lower_bound(2);
```

Points to:

```
first 2
```

---

upper_bound()

Definition:

```
First Element > x
```

Example:

```
1 2 2 2 5
```

```
ms.upper_bound(2);
```

Points to:

```
5
```

Count Occurrences

Most important multiset trick.

Input:

```
1 2 2 2 5
```

Count:

```
2
```

Formula:

```
distance(
    ms.lower_bound(2),
    ms.upper_bound(2)
);
```

Output:

```
3
```

Why not subtract?

Because:

```
multiset iterator
```

is not random-access.

So use:

```
distance()
```

Reverse Traversal

```
for(auto it=ms.rbegin();
    it!=ms.rend();
    it++)
{
    cout<<*it<<" ";
}
```

Empty

```
ms.empty();
```

Clear

```
ms.clear();
```

Swap

```
ms1.swap(ms2);
```

Time Complexity

Operation	Complexity
insert	O(log n)
erase	O(log n)
find	O(log n)
count	O(log n + k)
lower_bound	O(log n)
upper_bound	O(log n)

Set vs Multiset

Feature	Set	Multiset
Duplicates	No	Yes
Sorted	Yes	Yes
lower_bound	Yes	Yes
upper_bound	Yes	Yes

Common Interview Uses

Frequency Tracking

Instead of:

```
map<int,int>
```

Sometimes:

```
multiset<int>
```

is simpler.

Running Median

Very common.

Uses:

```
multiset
```

or

```
priority_queue
```

Sliding Window Median

LeetCode #480

Uses multiset.

Maintain Sorted Data With Duplicates

Perfect use case.

Example

Input:

```
5 1 3 3 2 3
```

Insert into multiset.

Stored:

```
1 2 3 3 3 5
```

Automatically sorted.

Duplicates preserved.

Practice Questions

Easy

Q1

Insert:

```
10 10 20 20 30
```

Print multiset.

Q2

Count occurrences of:

```
20
```

Q3

Delete one occurrence of:

```
10
```

Q4

Delete all occurrences of:

```
20
```

Q5

Print largest element.

Hint:

```
*ms.rbegin()
```

Medium

Q6

Find frequency using:

```
lower_bound
upper_bound
```

Q7

Find smallest element.

Q8

Find largest element.

Q9

Maintain running median.

Q10

Insert N numbers and answer frequency queries.

Interview Level

Q11

Sliding Window Median

LeetCode #480

Q12

Find Median From Data Stream

LeetCode #295

Q13

Contains Duplicate III

LeetCode #220

Common Mistakes

Mistake 1

```
ms.erase(10);
```

removes all 10s.

Many expect one removal.

Mistake 2

Using:

```
upper_bound - lower_bound
```

on multiset.

❌ Not valid.

Use:

```
distance()
```

Mistake 3

Thinking multiset is O(1).

No.

It's tree based.

```
O(log n)
```

Revision Sheet

Header

```
#include <set>
```

Declaration

```
multiset<int> ms;
```

Insert

```
ms.insert(x);
```

Remove One

```
ms.erase(ms.find(x));
```

Remove All

```
ms.erase(x);
```

Search

```
ms.find(x);
ms.count(x);
```

Bounds

```
ms.lower_bound(x);
ms.upper_bound(x);
```

Frequency

```
distance(
ms.lower_bound(x),
ms.upper_bound(x)
);
```

Utilities

```
ms.size();
ms.empty();
ms.clear();
ms.swap();
```

Complexity

```
insert      O(log n)
erase       O(log n)
find        O(log n)
count       O(log n)
```

Mental Trigger

When you see:

- Sorted duplicates
- Running median
- Sliding window median
- Frequency in sorted order
- Ordered duplicate storage

Think:

```
MULTISET
```

STL Progress

You've completed:

✅ Pair ✅ Vector ✅ Iterators ✅ Algorithms ✅ Stack ✅ Queue ✅ Deque ✅ Set ✅ Unordered Set ✅ Map ✅ Unordered Map ✅ Priority Queue ✅ Binary Search ✅ lower_bound ✅ upper_bound ✅ Multiset

This is already enough for most interviews.
